INDEX

Module 2: Android System Components & Data Handling

Sr. No	Title	Date	Signature
Menus, Dialogs and UI Interaction			
1	Develop an Android application implementing AppBar (Toolbar) and Options Menu.		
2	Develop an Android application demonstrating AlertDialog and custom DialogFragment.		
3	Develop an Android application implementing contextual menus and popup menus.		
Intents, Events and Listeners			
4	Develop an Android application implementing Explicit and Implicit Intents.		
5	Develop an Android application demonstrating Event Handling using Listeners (OnClick, OnLongClick, etc.).		
6	Develop an Android application using Adapters to populate Spinner or ListView dynamically.		
7	Develop an Android application implementing Started and Bound Services.		
8	Develop an Android application implementing Notifications using Notification Manager.		
9	Develop an Android application implementing Static and Dynamic Broadcast Receivers.		
Database and Networking			
10	Develop an Android application implementing SQLite database CRUD Operations.		
11	Develop an Android application integrating SQLite with ListView or RecyclerView.		
12	Develop an Android application to fetch and parse JSON data from a web service and display it in UI components.		
Concurrency, Media and Security			
13	Develop an Android application demonstrating Threads and Handlers for background processing.		
14	Develop an Android application implementing Media APIs and Telephone APIs.		
15	Develop an Android application implementing Runtime Permissions and security handling in AndroidManifest.xml.		

Practical 1

1. Develop an Android application implementing AppBar (Toolbar) and Options Menu.

Activitymain.xml

```
<LinearLayout
   xmlns:android="http://schemas.android.com/apk/res/android"
   xmlns:app="http://schemas.android.com/apk/res-auto"
   android:orientation="vertical"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   android:fitsSystemWindows="true">

   <androidx.appcompat.widget.Toolbar
      android:id="@+id/toolbar"
      android:layout_width="match_parent"
      android:layout_height="?attr/actionBarSize"
      android:background="#2196F3"
      android:titleTextColor="#FFFFFF"
      app:title="My App" />

   <TextView
      android:text="@string/welcome"
      android:textSize="25sp"
      android:gravity="center"
      android:layout_width="match_parent"
      android:layout_height="wrap_content"/>

</LinearLayout>
```

res/menu/main_menu.xml

```
<menu xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto">

<item
 android:id="@+id/action_home"
 android:title="Home"
 app:showAsAction="ifRoom" />

<item
 android:id="@+id/action_search"
 android:title="Search"
 app:showAsAction="ifRoom" />

<item
 android:id="@+id/action_settings"
 android:title="Settings"
 app:showAsAction="never" />

<item
 android:id="@+id/action_about"
 android:title="About"
 app:showAsAction="never" />
```

```
</menu>
```

MainActivity.kt

```
package com.example.relativelayout

import android.os.Bundle
import android.view.Menu
import android.view.MenuItem

import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val toolbar = findViewById<androidx.appcompat.widget.Toolbar>(R.id.toolbar)

        // Set properties BEFORE setSupportActionBar
        toolbar.title = "My App"
        toolbar.setTitleTextColor(android.graphics.Color.WHITE)
        toolbar.setBackgroundColor(android.graphics.Color.parseColor("#2196F3"))

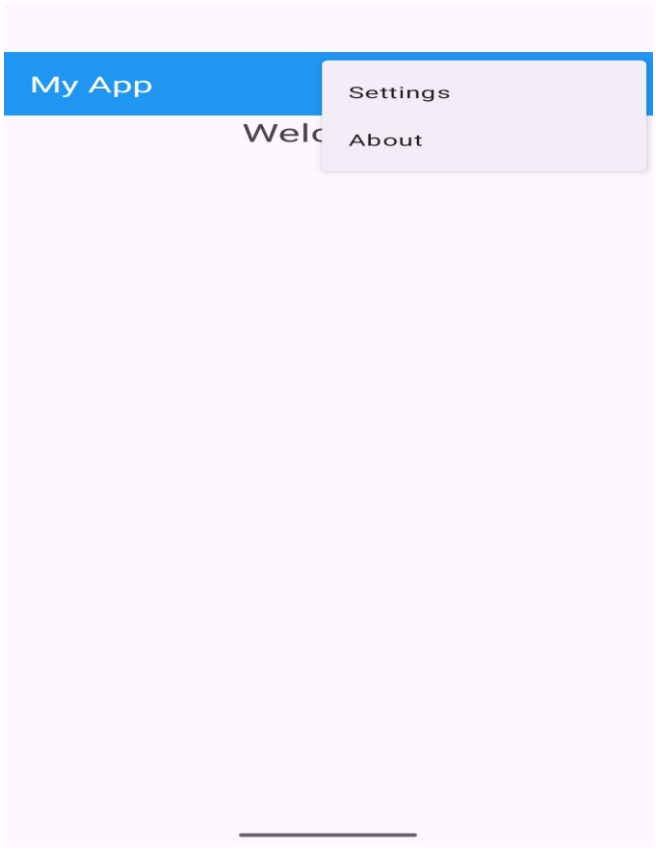
        setSupportActionBar(toolbar)
    }

    override fun onCreateOptionsMenu(menu: Menu): Boolean {
        menuInflater.inflate(R.menu.main_menu, menu)
        return true
    }

    override fun onOptionsItemSelected(item: MenuItem): Boolean {
        when (item.itemId) {
            R.id.action_home -> {
                Toast.makeText(this, "Home selected", Toast.LENGTH_SHORT).show()
                return true
            }
            R.id.action_search -> {
                Toast.makeText(this, "Search selected", Toast.LENGTH_SHORT).show()
                return true
            }
            R.id.action_settings -> {
                Toast.makeText(this, "Settings selected", Toast.LENGTH_SHORT).show()
                return true
            }
            R.id.action_about -> {
                Toast.makeText(this, "About selected", Toast.LENGTH_SHORT).show()
                return true
            }
        }
        return super.onOptionsItemSelected(item)
```

```
    }
}
```

Output



Toolbar and Options Menu were successfully implemented.

Practical 2

2. Develop an Android application demonstrating AlertDialog and custom DialogFragment.

Dialog

A dialog is a small window that prompts the user to make a decision or enter additional information. A dialog does not fill the screen and is normally used for modal events that require users to take an action before they can proceed.

activity_main.xml

```
<Button
android:text="Dialog"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:id="@+id/dialog"
android:textSize="30sp"
android:background="@color/colorPrimary"
android:onClick="dialog"

android:layout_alignParentStart="true"
android:layout_marginTop="210dp"
android:layout_marginStart="135dp"
android:layout_alignParentTop="true"/>
```

MainActivity.kt

```
class MainActivity : AppCompatActivity() {

override fun onCreate(savedInstanceState: Bundle?) {
super.onCreate(savedInstanceState)
setContentView(R.layout.activity_main)
}
fun dialog(view: View) {
var alert= AlertDialog.Builder(this@MainActivity)
alert.setTitle("Dialog Box")
alert.setMessage("Please Click on OK or cancel")
alert.setPositiveButton("OK"){
dialog, which ->
Toast.makeText(applicationContext,"You on Clicked Ok
button",Toast.LENGTH_SHORT).show()
}
alert.setNegativeButton("CANCEL"){
dialog, which ->
Toast.makeText(applicationContext,"You on Clicked Cancel
button",Toast.LENGTH_SHORT).show()
}
alert.show()

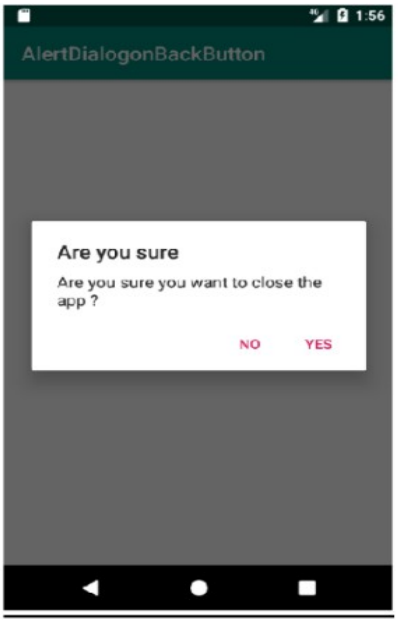
}
}
```



Example : Alertdialog on Back Button Pressed event

```
class MainActivity : AppCompatActivity() {
override fun onCreate(savedInstanceState: Bundle?) {
super.onCreate(savedInstanceState)
setContentView(R.layout.activity_main)
}
override fun onBackPressed() {
val builder = AlertDialog.Builder(this)
builder.setTitle("Are you sure")
builder.setMessage("Are you sure you want to close the app ?")
builder.setPositiveButton("Yes", {dialogInterface, i -> finish()})
builder.setNegativeButton("No",{ dialogInterface, i -> })
builder.show()
}
}
```

Output


1:56
AlertDialogonBackButton
Are you sure
Are you sure you want to close the app ?
NO
YES

Practical 3

3. Develop an Android application implementing contextual menus and popup menus.

Context Menu in Android

In Android, there are three types of menus available to define a set of options and actions in the Android apps. In Android, the context menu is like a floating menu and arises when the user has long-pressed or clicked on an item and is beneficial for implementing functions that define the specific content or reference frame effect.

Step By Step Implementation

Step 1: Create a New Project in Android Studio

Step 2: Working with the XML Files

Open res > Layout > activity_main.xml and write the following code. In this file add only a TextView to display a simple text.

Activity_main.xml

```
<androidx.constraintlayout.widget.ConstraintLayout
   xmlns:android="http://schemas.android.com/apk/res/android"
   xmlns:app="http://schemas.android.com/apk/res-auto"
   xmlns:tools="http://schemas.android.com/tools"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   android:background="@color/white"
   android:id="@+id/main"
   tools:context=".MainActivity">

   <TextView
      android:id="@+id/textView"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="Long press me!"
      android:textSize="20sp"
      android:textStyle="bold"
      app:layout_constraintBottom_toBottomOf="parent"
      app:layout_constraintEnd_toEndOf="parent"
      app:layout_constraintStart_toStartOf="parent"
      app:layout_constraintTop_toTopOf="parent"
      app:layout_constraintVertical_bias="0.4" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

Step 3: Working with the MainActivity file

Open the app > Java > Package > Mainactivity.kt file. In this step, add the code to show the ContextMenu. Whenever the app will start make a long click on a text and display the number of options to select of them for specific purposes. Comments are added inside the code to understand the code in more detail.

MainActivity.kt

```
package org.geeksforgeeks.demo

import android.graphics.Color
import android.os.Bundle
```

```
import android.view.ContextMenu
import android.view.ContextMenu.ContextMenuInfo
import android.view.MenuItem
import android.view.View
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity
import androidx.constraintlayout.widget.ConstraintLayout

class MainActivity : AppCompatActivity() {
    private lateinit var textView: TextView
    private lateinit var layout: ConstraintLayout

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        // Link those objects with their respective
        // id's that we have given in .XML file
        textView = findViewById(R.id.textView)
        layout = findViewById(R.id.main)

        // here you have to register a view for context
        // menu you can register any view like listview,
        // image view, textview, button etc
        registerForContextMenu(textView)
    }

    override fun onCreateContextMenu(menu: ContextMenu, v: View, menuInfo: ContextMenuInfo?)
{
        super.onCreateContextMenu(menu, v, menuInfo)

        // you can set menu header with title icon etc
        menu.setHeaderTitle("Choose a color")

        // add menu items
        menu.add(0, v.id, 0, "Yellow")
        menu.add(0, v.id, 0, "Gray")
        menu.add(0, v.id, 0, "Cyan")
    }

    // menu item select listener
    override fun onContextItemSelected(item: MenuItem): Boolean {
        if (item.title === "Yellow")
            layout.setBackgroundColor(Color.YELLOW)

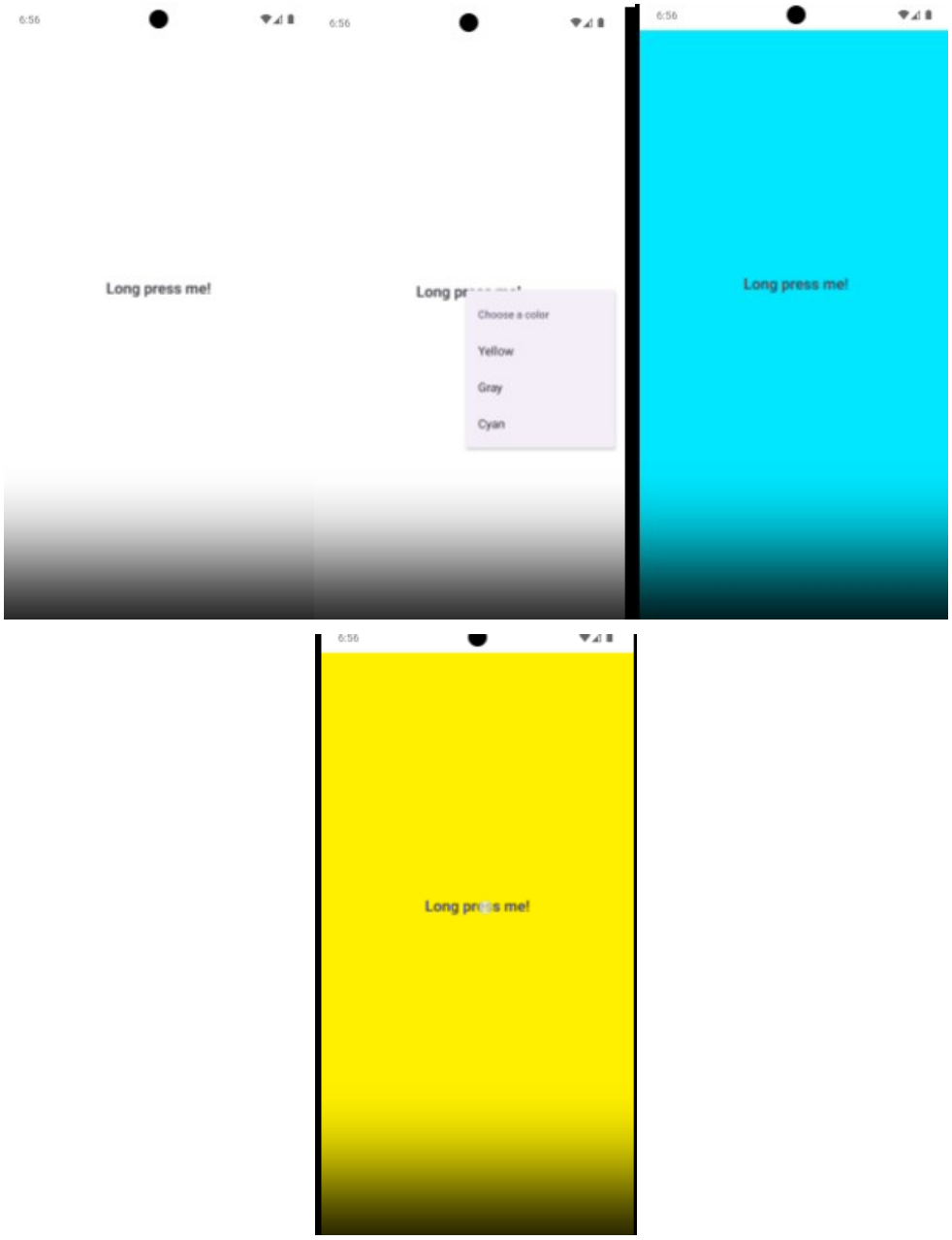
        else if (item.title === "Gray")
            layout.setBackgroundColor(Color.GRAY)

        else if (item.title === "Cyan")
            layout.setBackgroundColor(Color.CYAN)

        return true
    }
```

}

Output


6:56
Long press me!
6:56
Choose a color
Yellow
Gray
Cyan
6:56
Long press me!
6:56
Long press me!

Practical 4

4. Develop an Android application implementing Explicit and Implicit Intents.

Implicit Intent

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <EditText
        android:id="@+id/editText"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

    <Button
        android:id="@+id/btn"
        android:text="Search"
        android:onClick="search"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/editText" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

MainActivity.kt

```
import android.content.Intent
import android.net.Uri
import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import androidx.appcompat.app.AppCompatActivity

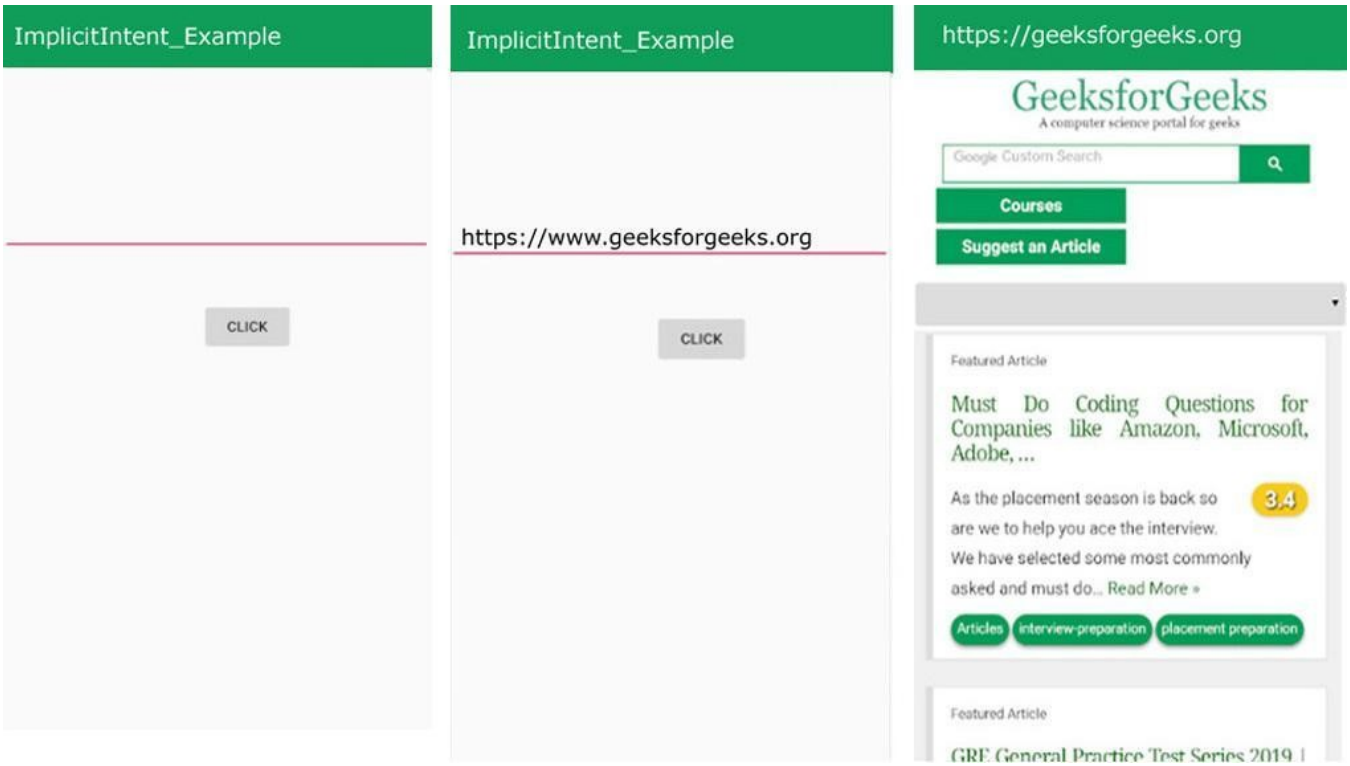
class MainActivity : AppCompatActivity() {

    lateinit var editText: EditText

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
```

```
        editText = findViewById(R.id.editText)
    }

    fun search() {
        val url = editText.text.toString()
        val urlIntent = Intent(Intent.ACTION_VIEW, Uri.parse(url))
        startActivity(urlIntent)
    }
}
```



Explicit Intent

Using explicit intent any other component can be specified. In other words, the targeted component is specified by explicit intent. So only the specified target component will be invoked.

Activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <TextView
        android:id="@+id/editText"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Welcome to GFG Home Screen"
        android:textAlignment="center"
        android:textSize="28sp"
```

```
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.0"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

    <Button
        android:id="@+id/btn1"
        android:text="Go to News Screen"
        android:onClick="newsScreen"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/editText" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

MainActivity.kt

```
import android.content.Intent
import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }

    fun newsScreen() {
        val i = Intent(applicationContext, MainActivity2::class.java)
        startActivity(i)
    }
}
```

Working with the activity_main2.xml File

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity2">

    <TextView
        android:id="@+id/editText"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Welcome to GFG News Screen"
        android:textAlignment="center"
        android:textSize="28sp"
        app:layout_constraintBottom_toBottomOf="parent"
```

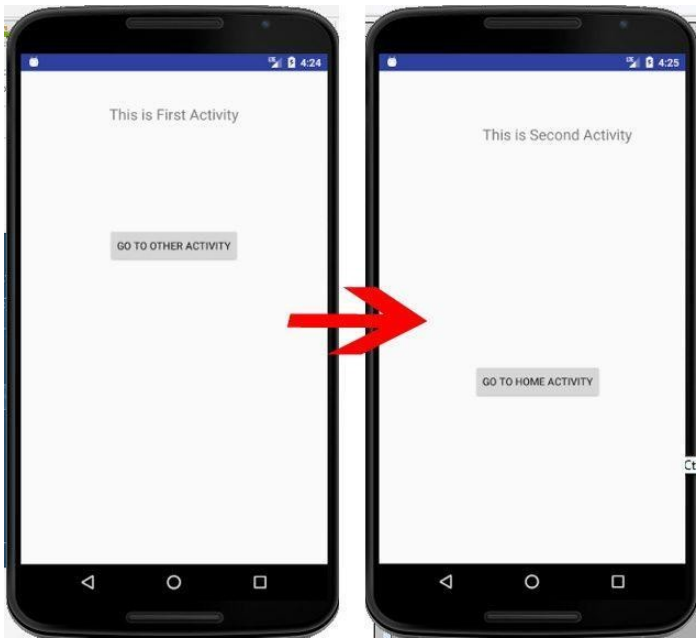
```
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.0"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
    <Button
        android:id="@+id/btn2"
        android:text="Go to Home Screen"
        android:onClick="homeScreen"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/editText" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

Working with the MainActivity2 File

```
import android.content.Intent
import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity

class MainActivity2 : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main2)
    }

    fun homeScreen() {
        val i = Intent(applicationContext, MainActivity::class.java)
        startActivity(i)
    }
}
```



Practical 5

5. Develop an Android application demonstrating Event Handling using Listeners (OnClick, OnLongClick, etc.).

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical"
android:padding="16dp">
<EditText
android:id="@+id/editText"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:hint="Type here"/>
<ImageView
android:id="@+id/imageView"
android:layout_width="200dp"
android:layout_height="200dp"
android:layout_gravity="center"
android:layout_marginTop="20dp"
android:src="@drawable/ic_launcher_background"/>

<Button
android:id="@+id/button"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="BUTTON"
android:layout_gravity="center"
android:layout_marginTop="20dp"/>

<TextView
android:id="@+id/textView"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:text="Event Messages"
android:layout_marginTop="20dp"
android:textSize="16sp"/>

</LinearLayout>
```

MainActivity.kt

```
class MainActivity : AppCompatActivity() {

lateinit var button: Button
lateinit var image: ImageView
lateinit var editText: EditText
lateinit var textView: TextView

override fun onCreate(savedInstanceState: Bundle?) {
super.onCreate(savedInstanceState)
setContentView(R.layout.activity_main)
```

```
button = findViewById(R.id.button)
image = findViewById(R.id.imageView)
editText = findViewById(R.id.editText)
textView = findViewById(R.id.textView)
// Button Click Listener
button.setOnClickListener {
textView.text = "You clicked on Button"
}

// Button Long Click Listener
button.setOnLongClickListener {
textView.text = "You long clicked on Button"
true
}
// Image Touch Listener
image.setOnTouchListener { v, event ->

when (event.action) {

MotionEvent.ACTION_DOWN -> {
textView.text = "ACTION DOWN"
}

MotionEvent.ACTION_UP -> {
textView.text = "ACTION UP"
}

MotionEvent.ACTION_POINTER_DOWN -> {
textView.text = "ACTION POINTER DOWN"
}
}
true
}
// EditText Enter Key Listener
editText.setOnKeyListener { v, keyCode, event ->

if (keyCode == KeyEvent.KEYCODE_ENTER && event.action == KeyEvent.ACTION_DOWN) {
textView.text = "Enter key pressed"
true
} else {
false
}}}}
```

Output


EventListener
hello
BUTTON
You long clicked on Button
You clicked on Button
ACTION UP
ACTION DOWN
ACTION UP
ACTION DOWN
ACTION UP
ACTION DOWN
ACTION UP

Practical 6

6. Develop an Android application using Adapters to populate Spinner or ListView dynamically.

Dynamic Spinner in Android

Activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<!--on below line we are creating our linear layout-->
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/idLLContainer"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_centerInParent="true"
    android:layout_gravity="center"
    android:gravity="center"
    android:orientation="vertical"
    tools:context=".MainActivity">
</LinearLayout>
```

Working with the MainActivity file

```
package com.gtappdevelopers.kotlingfgproject

import android.graphics.Typeface
import android.os.Bundle
import android.view.Gravity
import android.view.View
import android.view.ViewGroup
import android.widget.*
import androidx.appcompat.app.AppCompatActivity
import androidx.core.view.setPadding

class MainActivity : AppCompatActivity() {

    // on below line we are creating a variable.
    lateinit var languagesList: List<String>
    lateinit var containerLL: LinearLayout

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        // on below line we are initializing our variables
        languagesList = ArrayList()
        containerLL = findViewById(R.id.idLLContainer)

        // on below line we are adding languages
        // to our language list
        languagesList = languagesList + "Java"
        languagesList = languagesList + "Kotlin"
        languagesList = languagesList + "C++"
```

```
languagesList = languagesList + "C"
// on below line we are creating layout
// params for text view.
// and specifying width as match parent
// and height as wrap content
val txtLayoutParam = LinearLayout.LayoutParams(
    ViewGroup.LayoutParams.MATCH_PARENT,
    ViewGroup.LayoutParams.WRAP_CONTENT
)
// on below line we are adding gravity
txtLayoutParam.gravity = Gravity.CENTER

// on below line we are creating layout params for spinner.
// and specifying width as wrap parent and height as wrap content
val spinnerLayoutParam = LinearLayout.LayoutParams(
    ViewGroup.LayoutParams.WRAP_CONTENT,
    ViewGroup.LayoutParams.WRAP_CONTENT
)

// on below line we are adding gravity
spinnerLayoutParam.gravity = Gravity.CENTER

// on below line we are creating our dynamic text view
val headingTV = TextView(this)

// on the below line we are setting for our text view.
headingTV.text = "Dynamic Spinner in Android"

// on below line we are updating text size.
headingTV.textSize = 20f

// on below line we are updating text color and font
headingTV.setTextColor(resources.getColor(R.color.black))
headingTV.typeface = Typeface.DEFAULT_BOLD

// on below line we are adding padding
headingTV.setPadding(20)

// on below line we are specifying text alignment.
headingTV.textAlignment = TextView.TEXT_ALIGNMENT_CENTER

// on below line we are adding layout
// param for heading text view.
headingTV.layoutParams = txtLayoutParam

//create spinner programmatically
val spinner = Spinner(this)

// on below line we are adding params for spinner.
spinner.layoutParams = spinnerLayoutParam

// on below line we are adding our views
// to container linear layout
```

```
        containerLL.addView(headingTV)
        containerLL.addView(spinner)

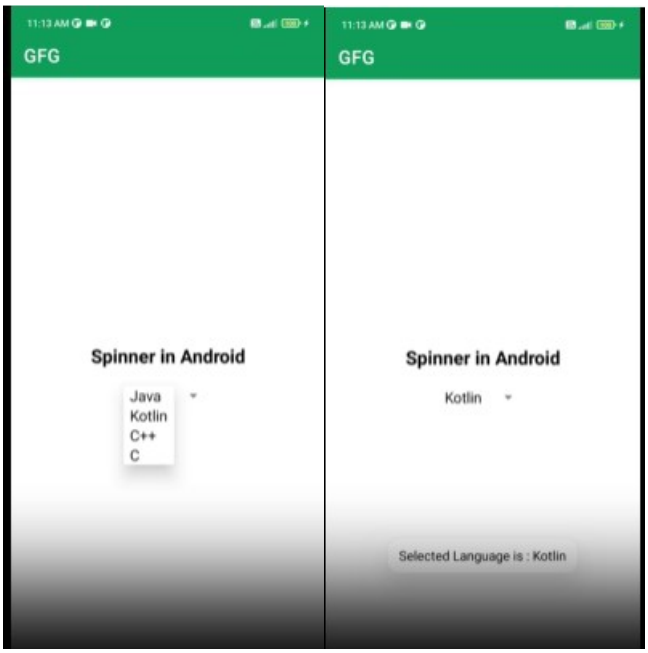
        // on below line we are checking
        // if spinner is not null
        if (spinner != null) {

            // on below line we are initializing
            // and setting our adapter
            // to our spinner.
            val adapter = ArrayAdapter(this, android.R.layout.simple_spinner_item, languagesList)
            spinner.adapter = adapter

            // on below line we are adding on item selected listener for spinner.
            spinner.onItemSelectedListener = object : AdapterView.OnItemSelectedListener {
                override fun onItemSelected(
                    parent: AdapterView<*>,
                    view: View, position: Int, id: Long
                ) {
                    // in on selected listener we are displaying a toast message
                    Toast.makeText(
                        this@MainActivity,
                        "Selected Language is : " +
                                "" + languagesList[position], Toast.LENGTH_SHORT
                    ).show()
                }

                override fun onNothingSelected(parent: AdapterView<*>) {
                }
            }
        }
    }}
```

Output



Practical 7

7. Develop an Android application implementing Started and Bound Services.

Example of Android Services

Playing music in the background is a very common example of services in android . From the time when a user starts the service, music play continuously in the background even if the user switches to another application. The user has to stop the service explicitly in order to pause the music. Below is the complete step-by-step implementation of this android service using a few callback methods.

Step 1: Create a new project

Step 2: Modify strings.xml file

All the strings which are used in the activity are listed in this file.

```
<resources>
<string name="app_name">Services_In_Android</string>
<string name="heading">Services In Android</string>
<string name="startButtonText">Start the Service</string>
<string name="stopButtonText">Stop the Service</string>
</resources>
```

Step 3: Working with the activity_main.xml file

Open the activity_main.xml file and add 2 Buttons in it which will start and stop the service. Below is the code for designing a proper activity layout.

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
   xmlns:android="http://schemas.android.com/apk/res/android"
   xmlns:app="http://schemas.android.com/apk/res-auto"
   xmlns:tools="http://schemas.android.com/tools"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   android:background="#168BC34A"
   tools:context=".MainActivity">

   <LinearLayout
      android:id="@+id/linearLayout"
      android:layout_width="match_parent"
      android:layout_height="wrap_content"
      android:layout_centerVertical="true"
      android:orientation="vertical"
      app:layout_constraintBottom_toBottomOf="parent"
      app:layout_constraintEnd_toEndOf="parent"
      app:layout_constraintStart_toStartOf="parent"
      app:layout_constraintTop_toTopOf="parent"
      app:layout_constraintVertical_bias="1.0"
      tools:ignore="MissingConstraints">

      <TextView
         android:id="@+id/textView1"
         android:layout_width="match_parent"
         android:layout_height="wrap_content"
         android:layout_marginBottom="170dp"
         android:fontFamily="@font/roboto"
```

```
        android:text="@string/heading"
        android:textAlignment="center"
        android:textAppearance="@style/TextAppearance.AppCompat.Large"
        android:textColor="@android:color/holo_green_dark"
        android:textSize="36sp"
        android:textStyle="bold" />

    <Button
        android:id="@+id/startButton"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:layout_marginStart="20dp"
        android:layout_marginTop="10dp"
        android:layout_marginEnd="20dp"
        android:layout_marginBottom="20dp"
        android:background="#4CAF50"
        android:fontFamily="@font/roboto"
        android:text="@string/startButtonText"
        android:textAlignment="center"
        android:textAppearance="@style/TextAppearance.AppCompat.Display1"
        android:textColor="#FFFFFF"
        android:textStyle="bold" />

    <Button
        android:id="@+id/stopButton"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:layout_marginStart="20dp"
        android:layout_marginTop="10dp"
        android:layout_marginEnd="20dp"
        android:layout_marginBottom="20dp"
        android:background="#4CAF50"
        android:fontFamily="@font/roboto"
        android:text="@string/stopButtonText"
        android:textAlignment="center"
        android:textAppearance="@style/TextAppearance.AppCompat.Display1"
        android:textColor="#FFFFFF"
        android:textStyle="bold" />

    <ImageView
        android:id="@+id/imageView"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="80dp"
        app:srcCompat="@drawable/banner" />
  </LinearLayout>

</androidx.constraintlayout.widget.ConstraintLayout>
```

Step 4: Creating the custom service class

```
import android.app.Service
import android.content.Intent
import android.media.MediaPlayer
```

```
import android.os.IBinder
import android.provider.Settings

class NewService : Service() {
    // declaring object of MediaPlayer
    private lateinit var player:MediaPlayer

    // execution of service will start
    // on calling this method
    override fun onStartCommand(intent: Intent, flags: Int, startId: Int): Int {

        // creating a media player which
        // will play the audio of Default
        // ringtone in android device
        player = MediaPlayer.create(this, Settings.System.DEFAULT_RINGTONE_URI)

        // providing the boolean
        // value as true to play
        // the audio on loop
        player.setLooping(true)

        // starting the process
        player.start()

        // returns the status
        // of the program
        return START_STICKY
    }

    // execution of the service will
    // stop on calling this method
    override fun onDestroy() {
        super.onDestroy()

        // stopping the process
        player.stop()
    }

    override fun onBind(intent: Intent): IBinder? {
        return null
    }
}
```

Step 5: Working with the MainActivity file

Now, the button objects will be declared and the process to be performed on clicking these buttons will be defined in the MainActivity class. Below is the code to implement this step.

```
import android.content.Intent
import android.os.Bundle
import android.view.View
import android.widget.Button
import androidx.appcompat.app.AppCompatActivity
```

```
class MainActivity : AppCompatActivity(), View.OnClickListener {

    // declaring objects of Button class
    private var start: Button? = null
    private var stop: Button? = null
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        // assigning ID of startButton
        // to the object start
        start = findViewById<View>(R.id.startButton) as Button

        // assigning ID of stopButton
        // to the object stop
        stop = findViewById<View>(R.id.stopButton) as Button

        // declaring listeners for the
        // buttons to make them respond
        // correctly according to the process
        start!!.setOnClickListener(this)
        stop!!.setOnClickListener(this)
    }

    override fun onClick(view: View) {

        // process to be performed
        // if start button is clicked
        if (view === start) {

            // starting the service
            startService(Intent(this, NewService::class.java))
        }

        // process to be performed
        // if stop button is clicked
        else if (view === stop) {

            // stopping the service
            stopService(Intent(this, NewService::class.java))
        }
    }
}
```

Step 6: Modify the AndroidManifest.xml file

To implement the services successfully on any android device, it is necessary to mention the created service in the AndroidManifest.xml file. It is not possible for a service to perform its task if it is not mentioned in this file. The service name is mentioned inside the application tag .

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.services_in_android">
```

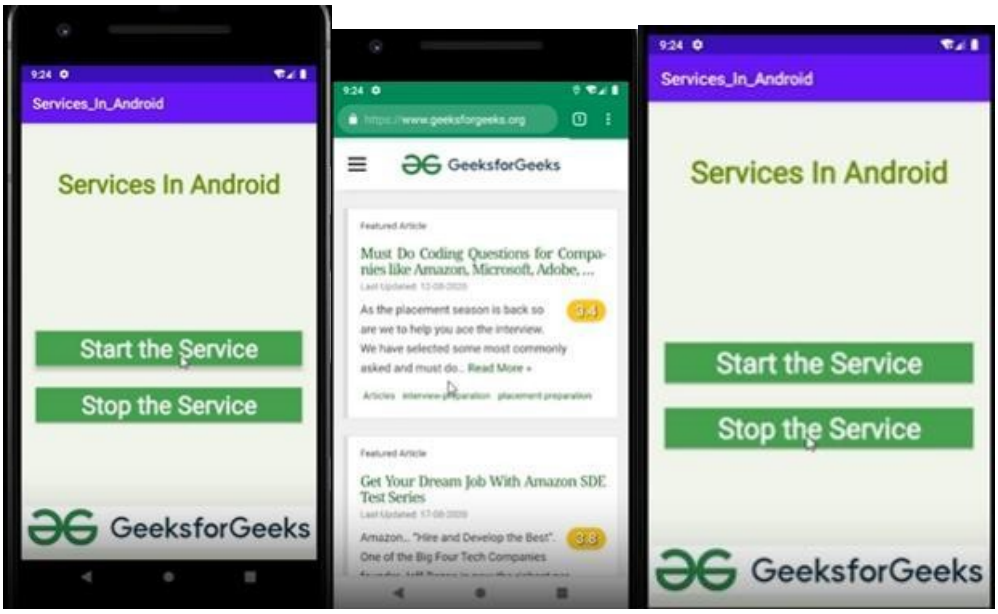
```
    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/AppTheme">
        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
        <meta-data
            android:name="preloaded_fonts"
            android:resource="@array/preloaded_fonts" />

        <!-- Mention the service name here -->
        <service android:name=".NewService"/>

    </application>

</manifest>
```

Output



Practical 8

8. Develop an Android application implementing Notifications using Notification Manager.

A notification is a message that Android displays outside your app's UI to provide the user with reminders, communication from other people, or other timely information from your app. Users can tap the notification to open your app or take an action directly from the notification.

activity_main.xml

```
<RelativeLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
tools:context=".MainActivity">
<Button
android:id="@+id/btn1"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Notification 1"
android:background="@color/colorPrimary"
android:layout_alignParentTop="true"
android:layout_marginTop="133dp"
android:onClick="show1"
android:layout_centerInParent="true"/>
<Button
android:id="@+id/btn2"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Notification 2"
android:background="@color/colorPrimary"
android:layout_marginTop="156dp"
android:layout_below="@+id/btn1"

android:onClick="show2"
android:layout_centerInParent="true"/>
</RelativeLayout>
```

MainActivity.kt

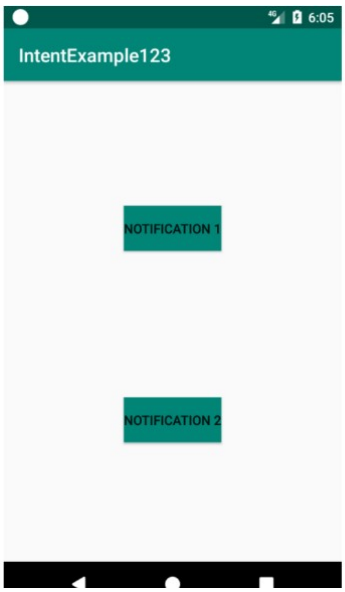
```
class MainActivity : AppCompatActivity() {
val CHANNEL_ID="123"
val NOTIFICATION_ID_1=111
val NOTIFICATION_ID_2=222
override fun onCreate(savedInstanceState: Bundle?) {
super.onCreate(savedInstanceState)
setContentView(R.layout.activity_main)
createchannel()
}
fun createchannel(){
if(Build.VERSION.SDK_INT>=Build.VERSION_CODES.O){
val channel= NotificationChannel(CHANNEL_ID,"notif",
NotificationManager.IMPORTANCE_DEFAULT)
val
notificationManager:NotificationManager=getSystemService(Context.NOTIFICATION_
```

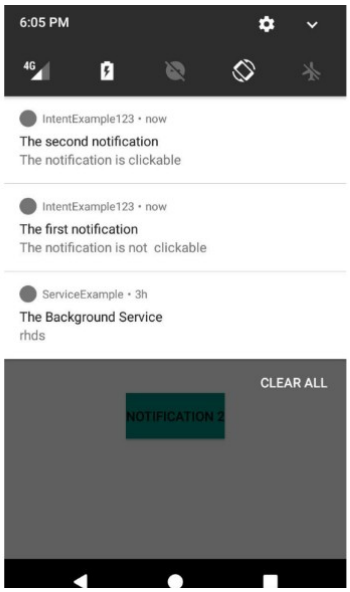
```
SERVICE) as NotificationManager
notificationManager.createNotificationChannel(channel)
}
}
fun show1(view: View){
var notif = NotificationCompat.Builder(this, CHANNEL_ID)
.setSmallIcon(R.drawable.notification_icon_background)
.setContentTitle("The first notification")
.setContentText("The notification is not clickable")
.setPriority(NotificationCompat.PRIORITY_DEFAULT)

var notificationManagerCompat = NotificationManagerCompat.from(this)
notificationManagerCompat.notify(NOTIFICATION_ID_1,notif.build())
}

fun show2(view: View){
val intent=Intent(this,MainActivity::class.java)
val pendingIntent= PendingIntent.getActivity(this,0,intent,0)

var notif = NotificationCompat.Builder(this, CHANNEL_ID)
.setSmallIcon(R.drawable.notification_icon_background)
.setContentTitle("The second notification")
.setContentText("The notification is clickable")
.setPriority(NotificationCompat.PRIORITY_DEFAULT)
.setContentIntent(pendingIntent)
.setAutoCancel(true)
var notificationManagerCompat = NotificationManagerCompat.from(this)
notificationManagerCompat.notify(NOTIFICATION_ID_2,notif.build())
}
}
```





Practical 9

9. Develop an Android application implementing Static and Dynamic Broadcast Receivers.

Broadcast in android is the system-wide events that can occur when the device starts, when a message is received on the device or when incoming calls are received, or when a device goes to airplane mode, etc. Broadcast Receivers are used to respond to these system-wide events. Broadcast Receivers allow us to register for the system and application events, and when that event happens, then the register receivers get notified. There are mainly two types of Broadcast Receivers:

- Static Broadcast Receivers: These types of Receivers are declared in the manifest file and works even if the app is closed.
- Dynamic Broadcast Receivers: These types of receivers work only if the app is active or minimized.

Since from API Level 26, most of the broadcast can only be caught by the dynamic receiver, so we have implemented dynamic receivers in our sample project given below. There are some static fields defined in the Intent class which can be used to broadcast different events. We have taken a change of airplane mode as a broadcast event, but there are many events for which broadcast register can be used. Following are some of the important system-wide generated intents:-

Step 1: Create a New Project

Step 2: Working with the activity_main.xml file

Go to the activity_main.xml file and refer to the following code. Below is the code for the activity_main.xml file.

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
   xmlns:android="http://schemas.android.com/apk/res/android"
   xmlns:tools="http://schemas.android.com/tools"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   tools:context=".MainActivity">
</androidx.constraintlayout.widget.ConstraintLayout>
```

Step 3: Working with the MainActivity file

Go to the MainActivity file and refer to the following code. Below is the code for the MainActivity file. Comments are added inside the code to understand the code in more detail.

```
import android.content.Intent
import android.content.IntentFilter
import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {

   // register the receiver in the main activity in order
   // to receive updates of broadcasts events if they occur
   lateinit var receiver: AirplaneModeChangeReceiver
   override fun onCreate(savedInstanceState: Bundle?) {
      super.onCreate(savedInstanceState)
      setContentView(R.layout.activity_main)

      receiver = AirplaneModeChangeReceiver()

      // Intent Filter is useful to determine which apps wants to receive
```

```
        // which intents,since here we want to respond to change of
        // airplane mode
        IntentFilter(Intent.ACTION_AIRPLANE_MODE_CHANGED).also {
            // registering the receiver
            // it parameter which is passed in registerReceiver() function
            // is the intent filter that we have just created
            registerReceiver(receiver, it)
        }
    }

    // since AirplaneModeChangeReceiver class holds a instance of Context
    // and that context is actually the activity context in which
    // the receiver has been created
    override fun onStop() {
        super.onStop()
        unregisterReceiver(receiver)
    }
}
```

Step 4: Create a new class

Go to app > java/kotlin > your package name(in which the MainActicity is present) > rightclick > New > Kotlin File/Class and name the files as AirplaneModeChangeReceiver . Below is the code for the AirplaneModeChangeReceiver file. Comments are added inside the code to understand the code in more detail.

```
import android.content.BroadcastReceiver
import android.content.Context
import android.content.Intent
import android.widget.Toast

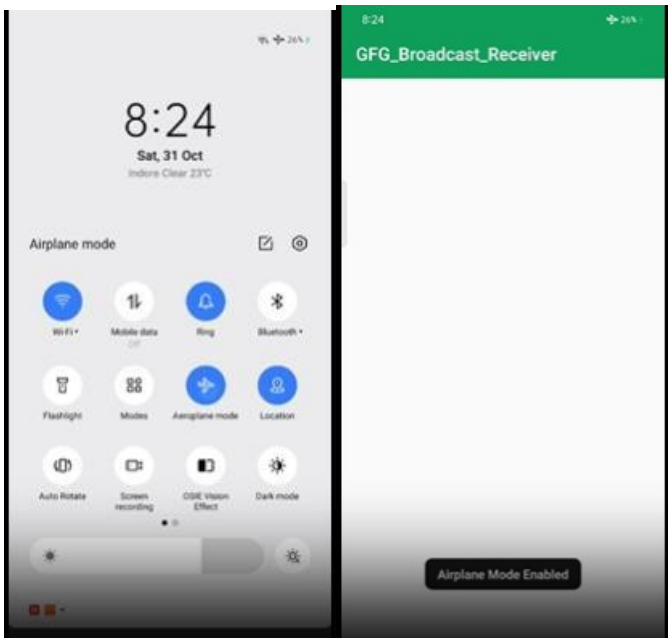
// AirplaneModeChangeReceiver class extending BroadcastReceiver class
class AirplaneModeChangeReceiver : BroadcastReceiver() {

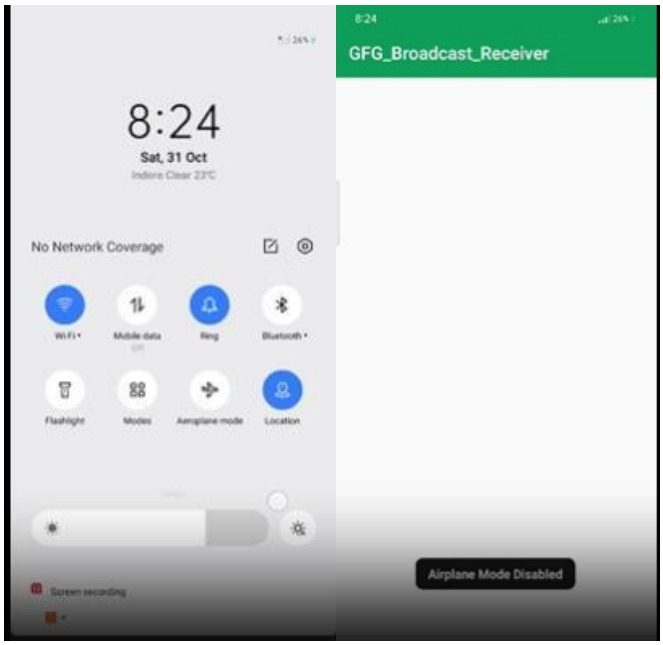
    // this function will be executed when the user changes his
    // airplane mode
    override fun onReceive(context: Context?, intent: Intent?) {

        // intent contains the information about the broadcast
        // in our case broadcast is change of airplane mode

        // if getBooleanExtra contains null value,it will directly return back
        val isAirplaneModeEnabled = intent?.getBooleanExtra("state", false) ?: return

        // checking whether airplane mode is enabled or not
        if (isAirplaneModeEnabled) {
            // showing the toast message if airplane mode is enabled
            Toast.makeText(context, "Airplane Mode Enabled", Toast.LENGTH_LONG).show()
        } else {
            // showing the toast message if airplane mode is disabled
            Toast.makeText(context, "Airplane Mode Disabled", Toast.LENGTH_LONG).show()
        }
    }
}
```


8:24
Sat, 31 Oct
Airplane mode
8:24
GFG_Broadcast_Receiver
Airplane Mode Enabled


8:24
Sat, 31 Oct
No Network Coverage
8:24
GFG_Broadcast_Receiver
Airplane Mode Disabled

Practical 10

10. Develop an Android application implementing SQLite database CRUD Operations.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout
  xmlns:android="http://schemas.android.com/apk/res/android"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  android:orientation="vertical"
  android:padding="20dp">

  <EditText
    android:id="@+id/etId"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Enter ID"
    android:inputType="number"/>

  <EditText
    android:id="@+id/etName"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Enter Name"/>

  <Button
    android:id="@+id/btnInsert"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Insert"/>

  <Button
    android:id="@+id/btnUpdate"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Update"/>

  <Button
    android:id="@+id/btnDelete"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Delete"/>

  <Button
    android:id="@+id/btnView"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="View Data"/>

  <TextView
    android:id="@+id/tvData"
```

```
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:padding="15dp"
    android:textSize="18sp"/>

</LinearLayout>
```

DBHelper.kt

```
package com.example.sqlitecrud

import android.content.ContentValues
import android.content.Context
import android.database.sqlite.SQLiteDatabase
import android.database.sqlite.SQLiteOpenHelper

class DBHelper(context: Context) :
    SQLiteOpenHelper(context, "StudentDB", null, 1) {

    override fun onCreate(db: SQLiteDatabase) {
        db.execSQL(
            "CREATE TABLE Student(id INTEGER PRIMARY KEY, name TEXT)"
        )
    }

    override fun onUpgrade(
        db: SQLiteDatabase,
        oldVersion: Int,
        newVersion: Int
    ) {
        db.execSQL("DROP TABLE IF EXISTS Student")
        onCreate(db)
    }

    fun insertData(id: Int, name: String): Boolean {
        val db = writableDatabase
        val values = ContentValues()
        values.put("id", id)
        values.put("name", name)

        return db.insert("Student", null, values) != -1L
    }

    fun updateData(id: Int, name: String): Boolean {
        val db = writableDatabase
        val values = ContentValues()
        values.put("name", name)

        return db.update(
            "Student",
            values,
            "id=?",
            arrayOf(id.toString())
        ) > 0
```

```
    }

    fun deleteData(id: Int): Boolean {
        val db = writableDatabase

        return db.delete(
            "Student",
            "id=?",
            arrayOf(id.toString())
        ) > 0
    }

    fun getData(): String {
        val db = readableDatabase
        val cursor = db.rawQuery("SELECT * FROM Student", null)

        var result = ""

        while (cursor.moveToNext()) {
            result += "ID: ${cursor.getInt(0)}\n"
            result += "Name: ${cursor.getString(1)}\n\n"
        }

        cursor.close()
        return result
    }
}
```

MainActivity.kt

```
package com.example.sqlitecrud

import android.os.Bundle
import android.widget.*
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {

    lateinit var db: DBHelper

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        db = DBHelper(this)

        val id = findViewById<EditText>(R.id.etId)
        val name = findViewById<EditText>(R.id.etName)
        val data = findViewById<TextView>(R.id.tvData)

        findViewById<Button>(R.id.btnInsert).setOnClickListener {
            if (db.insertData(
                    id.text.toString().toInt(),
                    name.text.toString()
```

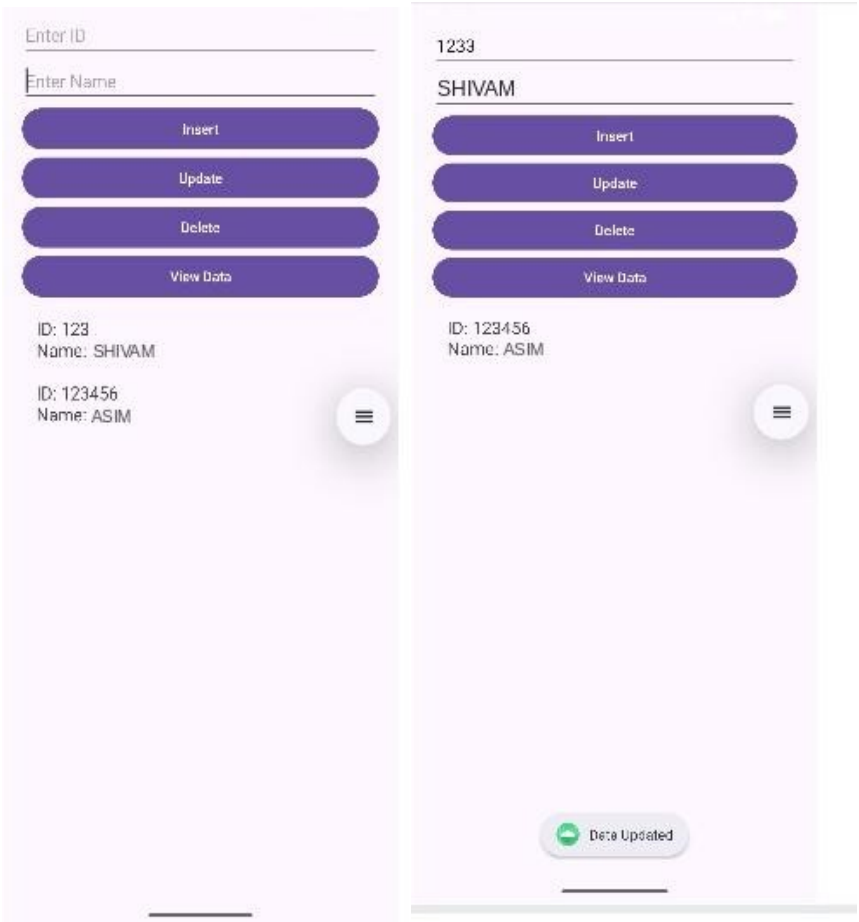
```
            )
        )
            Toast.makeText(this, "Data Inserted", Toast.LENGTH_SHORT).show()
    }

    findViewById<Button>(R.id.btnUpdate).setOnClickListener {
        if (db.updateData(
                id.text.toString().toInt(),
                name.text.toString()
            )
        )
            Toast.makeText(this, "Data Updated", Toast.LENGTH_SHORT).show()
    }

    findViewById<Button>(R.id.btnDelete).setOnClickListener {
        if (db.deleteData(id.text.toString().toInt()))
            Toast.makeText(this, "Data Deleted", Toast.LENGTH_SHORT).show()
    }

    findViewById<Button>(R.id.btnView).setOnClickListener {
        data.text = db.getData()
    }
  }
}
```

Output:



Practical 11

11. Develop an Android application integrating SQLite with ListView or RecyclerView.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout
   xmlns:android="http://schemas.android.com/apk/res/android"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   android:orientation="vertical"
   android:padding="20dp">

   <EditText
      android:id="@+id/etName"
      android:layout_width="match_parent"
      android:layout_height="wrap_content"
      android:hint="Enter Student Name"/>

   <Button
      android:id="@+id/btnAdd"
      android:layout_width="match_parent"
      android:layout_height="wrap_content"
      android:text="Add Student"/>

   <ListView
      android:id="@+id/listView"
      android:layout_width="match_parent"
      android:layout_height="match_parent"/>

</LinearLayout>
```

DBHelper.kt

```
package com.example.sqlitecrud

import android.content.ContentValues
import android.content.Context
import android.database.sqlite.SQLiteDatabase
import android.database.sqlite.SQLiteOpenHelper

class DBHelper(context: Context) :
   SQLiteOpenHelper(context, "StudentDB", null, 1) {

   override fun onCreate(db: SQLiteDatabase) {
      db.execSQL(
         "CREATE TABLE Student(id INTEGER PRIMARY KEY AUTOINCREMENT, name
TEXT)"
      )
   }

   override fun onUpgrade(
      db: SQLiteDatabase,
```

```
        oldVersion: Int,
        newVersion: Int
    ) {
        db.execSQL("DROP TABLE IF EXISTS Student")
        onCreate(db)
    }

    fun addStudent(name: String) {
        val db = writableDatabase
        val values = ContentValues()
        values.put("name", name)

        db.insert("Student", null, values)
    }

    fun getStudents(): ArrayList<String> {
        val list = ArrayList<String>()
        val db = readableDatabase

        val cursor = db.rawQuery(
            "SELECT name FROM Student",
            null
        )

        while (cursor.moveToNext()) {
            list.add(cursor.getString(0))
        }

        cursor.close()
        return list
    }}
```

MainActivity.kt

```
package com.example.sqlitecrud
import android.os.Bundle
import android.widget.*
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {

    lateinit var db: DBHelper
    lateinit var listView: ListView
    lateinit var adapter: ArrayAdapter<String>
    var studentList = ArrayList<String>()

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        db = DBHelper(this)

        val name = findViewById<EditText>(R.id.etName)
        val button = findViewById<Button>(R.id.btnAdd)
```

```
        listView = findViewById(R.id.listView)

        studentList = db.getStudents()

        adapter = ArrayAdapter(
            this,
            android.R.layout.simple_list_item_1,
            studentList
        )

        listView.adapter = adapter

        button.setOnClickListener {

            val studentName = name.text.toString()

            if (studentName.isNotEmpty()) {

                db.addStudent(studentName)

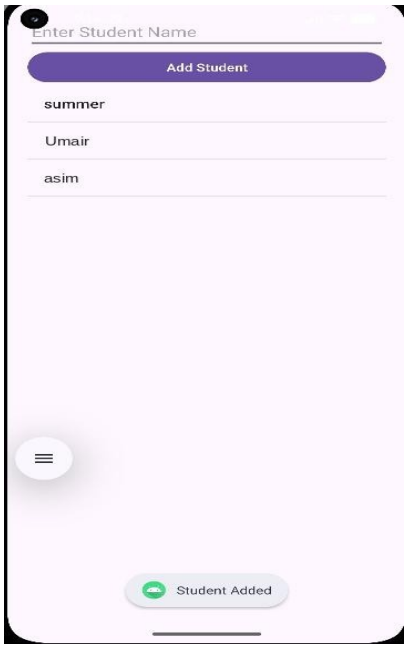
                studentList.clear()
                studentList.addAll(db.getStudents())

               adapter.notifyDataSetChanged()

                 name.text.cle ar()

                Toast.makeText(
                this,
           "Student Added",
         Toast.LENGTH_SHORT ).show()
}}}}
```

Output



Practical 12

12. Develop an Android application to fetch and parse JSON data from a web service and display it in UI components.

1. Add Internet Permission

Open AndroidManifest.xml and add this above <application>:

```
<uses-permission android:name="android.permission.INTERNET"/>
<?xml version="1.0" encoding="utf-8"?>

<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <uses-permission android:name="android.permission.INTERNET"/>

    <application
        android:theme="@style/Theme.SQLiteCRUD"
        android:label="JSON App">

        <activity
            android:name=".MainActivity"
            android:exported="true">

            <intent-filter>
                <action android:name="android.intent.action.MAIN"/>
                <category android:name="android.intent.category.LAUNCHER"/>
            </intent-filter>

        </activity>

    </application>

</manifest>
```

Activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="20dp">

    <Button
        android:id="@+id/btnFetch"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Fetch JSON Data"/>

    <TextView
        android:id="@+id/tvData"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
```

```
        android:padding="20dp"
        android:textSize="18sp"/>

</LinearLayout>
```

MainActivity.kt

```
package com.example.jsonapp

import android.os.Bundle
import android.widget.*
import androidx.appcompat.app.AppCompatActivity
import org.json.JSONObject
import java.net.URL
import kotlin.concurrent.thread

class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val button = findViewById<Button>(R.id.btnFetch)
        val textView = findViewById<TextView>(R.id.tvData)

        button.setOnClickListener {

            thread {

                try {
                    val url =
                        URL("https://jsonplaceholder.typicode.com/users/1")

                    val response = url.readText()

                    val jsonObject = JSONObject(response)

                    val name = jsonObject.getString("name")
                    val email = jsonObject.getString("email")

                    runOnUiThread {

                        textView.text =
                            "Name: $name\n\nEmail: $email"
                    }

                } catch (e: Exception) {

                    runOnUiThread {
                        textView.text = "Error: ${e.message}"
                    }
                }
            }
        }
```

```
    }
}
```

Output



Practical 13

13. Develop an Android application demonstrating Threads and Handlers for background processing.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout
   xmlns:android="http://schemas.android.com/apk/res/android"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   android:orientation="vertical"
   android:gravity="center"
   android:padding="20dp">

   <TextView
      android:id="@+id/tvStatus"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="Ready"
      android:textSize="20sp"/>

   <Button
      android:id="@+id/btnStart"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="Start Task"
      android:layout_marginTop="20dp"/>

</LinearLayout>
```

MainActivity.kt

```
package com.example.threadhandler

import android.os.Bundle
import android.os.Handler
import android.os.Looper
import android.widget.Button
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {

   override fun onCreate(savedInstanceState: Bundle?) {
      super.onCreate(savedInstanceState)
      setContentView(R.layout.activity_main)

      val status = findViewById<TextView>(R.id.tvStatus)
      val button = findViewById<Button>(R.id.btnStart)

      val handler = Handler(Looper.getMainLooper())
```

```
        button.setOnClickListener {

            status.text = "Processing..."

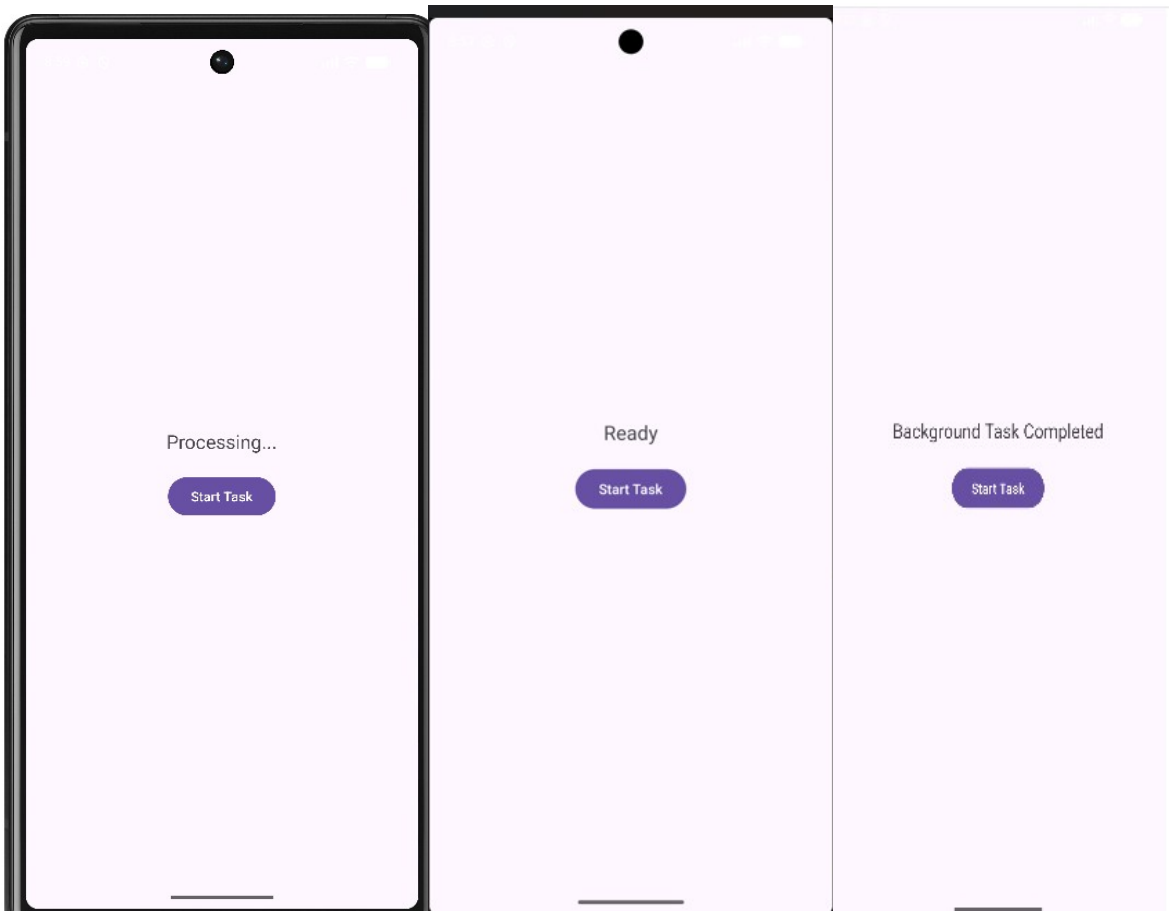
            Thread {

                Thread.sleep(3000)

                handler.post {
                    status.text = "Background Task Completed"
                }

            }.start()
        }
    }
}
```

Output



Practical 14

14. Develop an Android application implementing Media APIs and Telephone APIs.

Programming Media API and Telephone API

Media API

Provides classes that manage various media interfaces in audio and video.

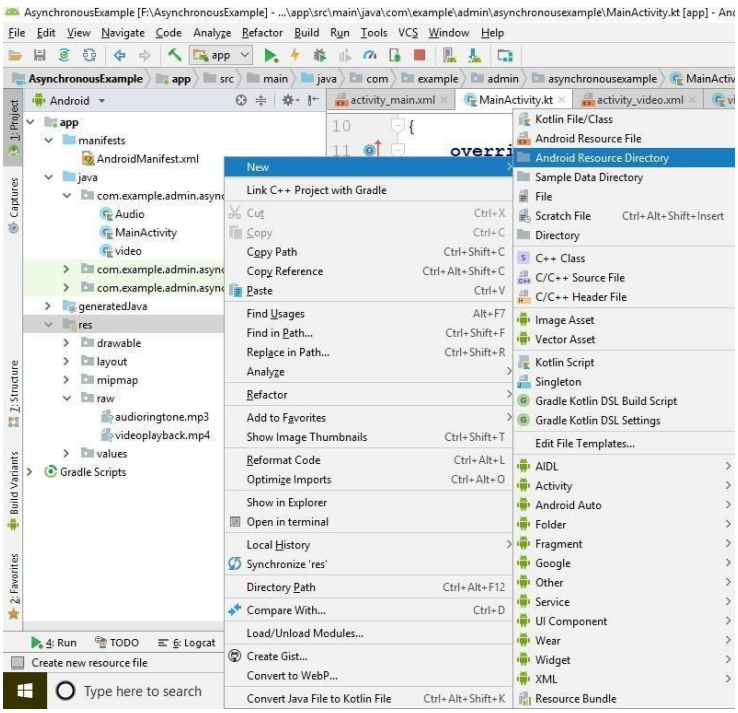
The Media APIs are used to play and, in some cases, record media files. This includes audio (e.g., play MP3s or other music files, ringtones, game sound effects, or DTMF tones) and video (e.g., play a video streamed over the web or from local storage). Activity_main.xml

```
<RelativeLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
tools:context=".MainActivity">

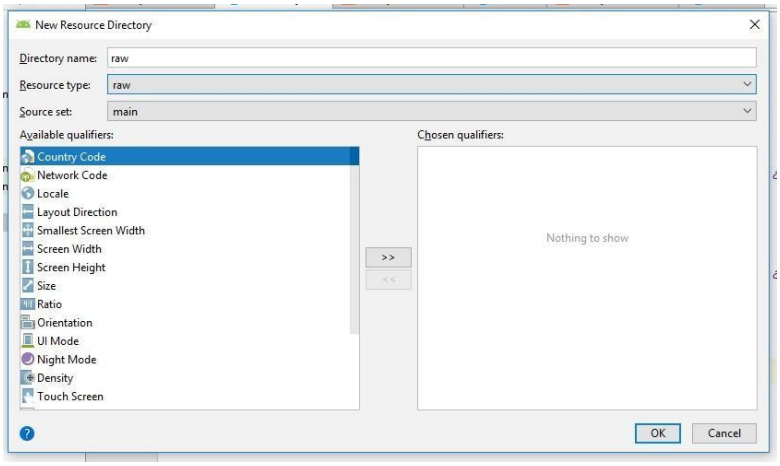
<Button
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:id="@+id/audio"
android:layout_alignParentTop="true"
android:text="Audio"
android:background="@color/colorPrimary"
android:layout_centerHorizontal="true"
android:layout_marginTop="150dp"
android:onClick="audio"
/>

<Button
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:id="@+id/video"
android:text="video"
android:background="@color/colorPrimary"
android:layout_centerHorizontal="true"
android:layout_below="@+id/audio"
android:layout_marginTop="60dp"
android:onClick="video"
/>
</RelativeLayout>
```

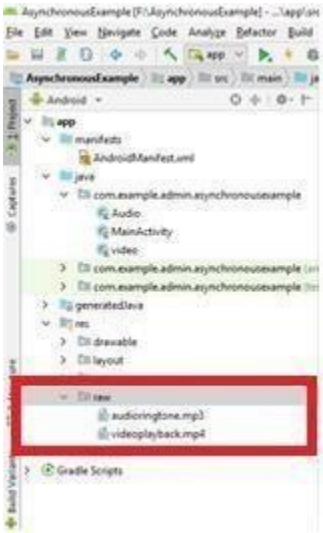
Create android resource directory under res folder to store audio or video file



Change the resource type to raw



Paste the audio (.mp3) / video(.mp4) file in the raw folder



activity_audio.xml

<RelativeLayout

```
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".Audio">

<SeekBar
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:id="@+id/audiobar"
        android:layout_alignParentTop="true"
        android:layout_marginTop="130dp"/>
<Button android:id="@+id/play"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Play"
        android:background="@color/colorPrim
        ary"
        android:layout_below="@id/audiobar"
        android:layout_alignParentLeft="true"
        android:layout_marginLeft="40dp"
        android:layout_marginTop="150dp"
        android:layout_marginRight="70dp"
        android:onClick="play"
/>
<Button android:id="@+id/stop"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Stop"
        android:background="@color/colorPrim
        ary"
        android:layout_below="@id/audiobar"
        android:layout_alignParentRight="true"
        android:layout_toRightOf="@+id/play"
        android:layout_marginRight="40dp"
        android:layout_marginLeft="70dp"
        android:layout_marginTop="150dp"
        android:onClick="stop"
/>
<Button android:id="@+id/back"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="BACK"
        android:background="@color/colorPrim
        ary" android:layout_below="@+id/play"
        android:layout_marginTop="50dp"
        android:layout_centerHorizontal="true"
        android:onClick="back"
/>
</RelativeLayout>
```

Audio.kt

```
class Audio : AppCompatActivity() {
```

```
lateinit var mediaPlayer:
MediaPlayer lateinit var handler:
Handler lateinit var runnable:
Runnable
    @RequiresApi(Build.VERSION_CODES.LOLLIPOP) override fun
    onCreate(savedInstanceState: Bundle?) { super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_audio) mediaPlayer =
    MediaPlayer.create(applicationContext, R.raw.audioringtone) var attributes =
    AudioAttributes.Builder()
            .setContentType(AudioAttributes.CONTENT_TYPE_MUSIC)
            .build()
        mediaPlayer.setAudioAttributes(attributes)
        handler = Handler()
        mediaPlayer.setOnPreparedListener(object :
        MediaPlayer.OnPreparedListener { override fun
        onPrepared(mp: MediaPlayer?) { audiobar.max
        = mediaPlayer.duration changeprogress()
        mediaPlayer.start()

            } })
        audiobar.setOnSeekBarChangeListener(object :
SeekBar.OnSeekBarChangeListener { override fun onProgressChanged(seekBar:
SeekBar?, progress: Int, input: Boolean) {
//
    Toast.makeText(applicationContext,progress.toString(),Toast.LENGTH_SHORT).show
()
                if (input) {
                    mediaPlayer.seekTo(progress)
                }
            }
            override fun onStartTrackingTouch(seekBar: SeekBar?) {
            }
            override fun onStopTrackingTouch(seekBar: SeekBar?) {
            }
        }) }
    fun changeprogress() { audiobar.progress =
        mediaPlayer.currentPosition if
        (mediaPlayer.isPlaying) { runnable = Runnable
        { changeprogress()
            }
            handler.postDelayed(runnable, 1000)
        }
    }
    fun play(view: View) { audiobar.progress =
        mediaPlayer.currentPosition
        mediaPlayer.start()
    }
    fun stop(view: View) {
        mediaPlayer.pause()
    }
    fun back(view:View){
        startActivity(Intent(this,MainActivity::class.java))
    }
```

```
    override fun onResume() {
        super.onResume() mediaPlayer.start()
        audiobar.progress =
        mediaPlayer.currentPosition
    }
    override fun onPause() {
        super.onPause()
        mediaPlayer.pause()
    }
    override fun onDestroy() {
        super.onDestroy()
        mediaPlayer.release()
        handler.removeCallbacks(runnable)
    }
}
```

activity_video.xml

```
<RelativeLayout
        xmlns:android="http://schemas.android.com/apk/res/android
        " xmlns:tools="http://schemas.android.com/tools"
        xmlns:app="http://schemas.android.com/apk/res-auto"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        tools:context=".video">

    <VideoView
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:id="@+id/videoView"/>
</RelativeLayout>
```

video.kt

```
 class video :
AppCompatActivity() {

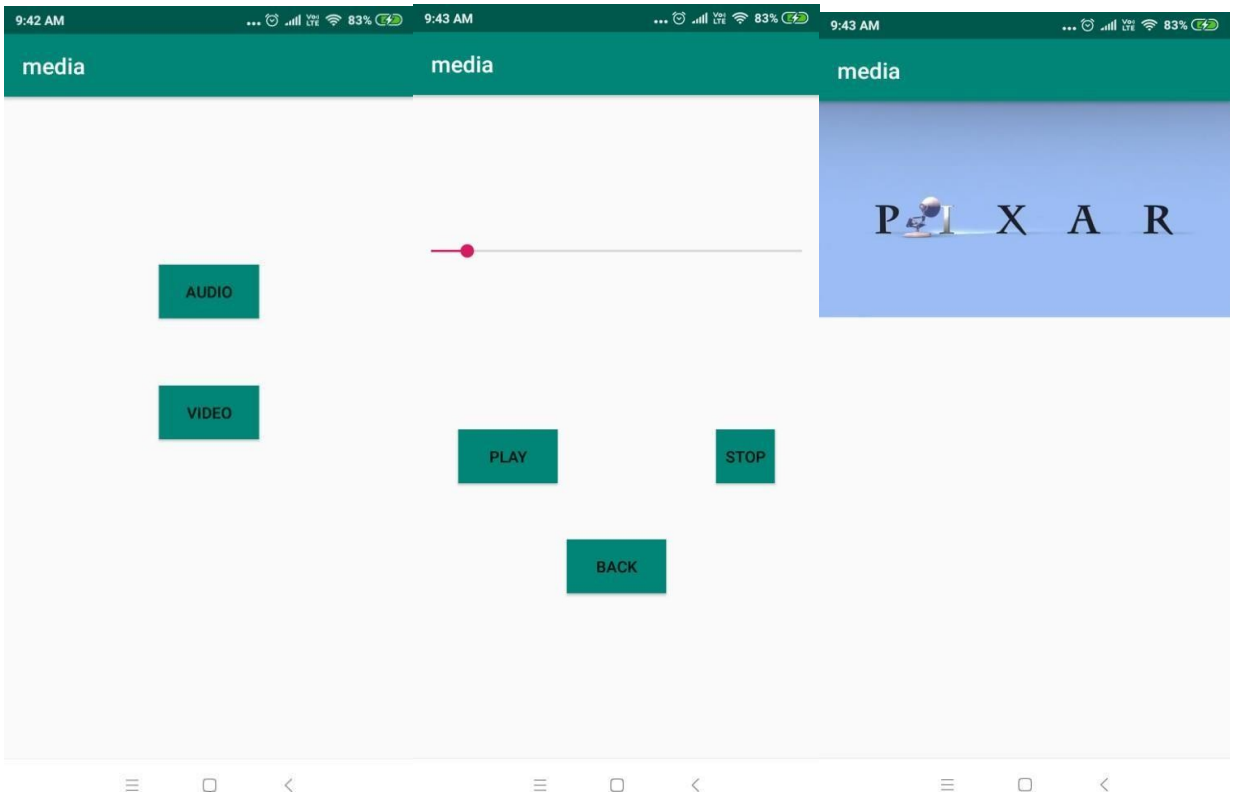
    override fun onCreate(savedInstanceState: Bundle?)
        { super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_video) var
        mediaController=MediaController(this) var
path="android.resource://com.example.admin.asynchronousexample/${R.raw.videopla
yback}" var uri= Uri.parse(path) videoView.setVideoURI(uri)
mediaController.setAnchorView(videoView)
videoView.setMediaController(mediaController) videoView.start()
    }
}
```

Note: In above kt file (Video.kt) : var path="android.resource://com.example.admin.asynchronousexample/${R.raw.videoplayback}" this path (com.example.admin.asynchronousexample)should be as per your package name

MainActivity.kt

```
class MainActivity :
AppCompatActivity()
```

```
{
    override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState) setContentView(R.layout.activity_main)
}
fun audio(view: View){ var intent=
    Intent(this,Audio::class.java)
    startActivity(intent)
}
fun video(view:View){ var intent=
    Intent(this,video::class.java)
    startActivity(intent)
}
}
```



Telephone API

The Telephony provider contains data related to phone operation, specifically SMS and MMS messages, access to the APN list, including the MMSC to use, and the service state.

You can access contacts using contact class you can make a calland many more functions are available in telephone api

Add the follwing permissions in androidmanifest.xml file in the manifest tag

```
<uses-permission android:name="android.permission.READ_CONTACTS" >
    </uses-permission>
    <uses-permission android:name="android.permission.CALL_PHONE"> </uses-
    permission>
```

Activity_main.xml

```
<RelativeLayout
        xmlns:android="http://schemas.android.com/apk/res/android"
        xmlns:tools="http://schemas.android.com/tools"
        xmlns:app="http://schemas.android.com/apk/res-auto"
```

```
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        tools:context=".MainActivity">
    <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:id="@+id/title" android:text="Click
            on number to make a call"
            android:textSize="25sp"
            android:textAlignment="center"
            android:textStyle="bold"
    />
    <ListView
            android:id="@+id/contacts"
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:layout_below="@+id/title"
    >
    </ListView>
</RelativeLayout>
```

Create a new xml resource file for inflating in listadapter and rename it (contact_data.xml)

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
              android:orientation="vertical" android:layout_width="match_parent"
              android:layout_height="match_parent">
    <TextView android:id="@+id/cdata"
            android:layout_width="match_pa
            rent"
            android:layout_height="wrap_co
            ntent" android:textSize="25sp"/>
</LinearLayout>
```

Create a model class from storing the contact details like name,phone number etc contact.kt

```
class contact { var
    pno:String?=nul
    l var
    name:String?=
    null fun
    getContact():St
    ring
    {
        return "$name \n$pno"
    }
}
```

Create a listadapter class for storing the contact details in list

listadapter.kt

```
class listadapter(var activity:Activity, var contactlist:ArrayList<contact>):BaseAdapter(){

    override fun getView(position: Int, convertView: View?, parent: ViewGroup?): View { var

    inflater=LayoutInflater.from(activity)
```

```
        var view=inflater.inflate(R.layout.contact_data,null)
        view.cdata.text=contactlist[position].getContact()
        view.setOnClickListener { var
            alertDialog=AlertDialog.Builder(activity)
            alertDialog.setTitle("Make a call")
  alertDialog.setMessage("to:${contactlist[position].name}\n${contactlist[position].pno}")
            alertDialog.setPositiveButton("Yes",DialogInterface.OnClickListener { dialog,
                    which ->
                try {
                    var intent = Intent(Intent.ACTION_CALL)
                    intent.data = Uri.parse("tel:${contactlist[position].pno}")
                    activity.startActivity(intent)
                }
                catch(e:Exception){
                   Toast.makeText(activity,"Please allow the permission" +
                           "",Toast.LENGTH_SHORT).show()
                }
            })
            alertDialog.setNegativeButton("No",DialogInterface.OnClickListener { dialog, which
->
                dialog.cancel()
            })
            alertDialog.create()
            alertDialog.show()
        }
        return view
    }
    override fun getItem(position: Int): Any {
```

```
        return contactlist[position]
    }
    override fun getItemId(position: Int): Long {
        return position.toLong()
    }
    override fun getCount(): Int {
        return contactlist.size
    }
}
```

MainActivity.kt

```
class MainActivity : AppCompatActivity() {
    lateinit var adapter:listadapter
    lateinit var contactList:ArrayList<contact>
    var cursor: Cursor?=null
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState) setContentView(R.layout.activity_main)
        var permissionCode1= ContextCompat.checkSelfPermission(this,
Manifest.permission.READ_CONTACTS)
        var permissionCode2= ContextCompat.checkSelfPermission(this,
Manifest.permission.CALL_PHONE)
        if(permissionCode1==PackageManager.PERMISSION_GRANTED &&
permissionCode2==PackageManager.PERMISSION_GRANTED)
        {
            showContacts()
        }
        else{
            ActivityCompat.requestPermissions(this,
arrayOf(Manifest.permission.READ_CONTACTS,Manifest.permission.CALL_PHONE),
100)
```

```
        }

        adapter= listadapter(this,contactList)
        contacts.adapter=adapter

    }

        override fun onRequestPermissionsResult(requestCode: Int, permissions: Array<out
String>, grantResults: IntArray) { super.onRequestPermissionsResult(requestCode,

        permissions, grantResults) if(requestCode==100){ showContacts()

        }

        else{

            Toast.makeText(this,"Please Provide the Contact
permission",Toast.LENGTH_SHORT).show()

        }

    }

    private fun showContacts() {

cursor=contentResolver.query(ContactsContract.CommonDataKinds.Phone.CONTEN
T_URI, null,null,null,ContactsContract.Contacts.SORT_KEY_PRIMARY )

        contactList=ArrayList<contact>
        () var number: String?

        var lastnumber:String?=null

        while(cursor!!.moveToNext())

        {

number=(cursor!!.getString(cursor!!.getColumnIndex(ContactsContract.CommonDataKi
nds.Phone.NUMBER)))
            if(number!=null ) {

                number=number.replace("\\s".toRegex(),

                "") if (!number!!.equals(lastnumber)) {

                    lastnumber = number

                    var contact = contact()

                    contact.name =

cursor!!.getString(cursor!!.getColumnIndex(ContactsContract.CommonDataKinds.Phon
e.DISPLAY_NAME))
```

```
contact.pno =
cursor!!.getString(cursor!!.getColumnIndex(ContactsContract.CommonDataKinds.Phon
e.NUMBER))

                    contactList.add(contact)

                }

            }

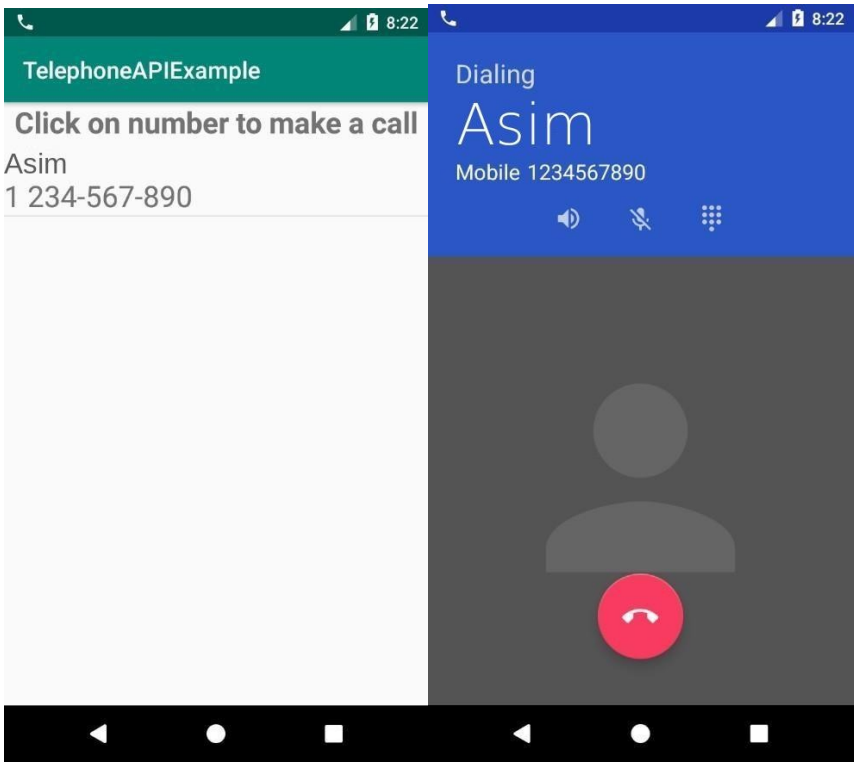
        }

        cursor!!.close()

    }

}
```

Output:



Practical 15

15. Develop an Android application implementing Runtime Permissions and security handling in AndroidManifest.xml

1. AndroidManifest.xml

Add the camera permission outside the <application> tag:

```
<?xml version="1.0" encoding="utf-8"?>

<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <uses-permission android:name="android.permission.CAMERA"/>

    <application
        android:allowBackup="false"
        android:label="Permission App"
        android:supportsRtl="true"
        android:theme="@style/Theme.PermissionApp">

        <activity
            android:name=".MainActivity"
            android:exported="true">

            <intent-filter>
                <action android:name="android.intent.action.MAIN"/>
                <category android:name="android.intent.category.LAUNCHER"/>
            </intent-filter>

        </activity>

    </application>

</manifest>
```

2. activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="20dp">

    <Button
        android:id="@+id/btnPermission"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Request Camera Permission"/>

    <TextView
        android:id="@+id/tvStatus"
```

```
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Permission not granted"
        android:textSize="18sp"
        android:layout_marginTop="20dp"/>

</LinearLayout>
```

3. MainActivity.kt

```
package com.example.permissionapp

import android.Manifest
import android.content.pm.PackageManager
import android.os.Bundle
import android.widget.*
import androidx.appcompat.app.AppCompatActivity
import androidx.core.app.ActivityCompat
import androidx.core.content.ContextCompat

class MainActivity : AppCompatActivity() {

    private val CAMERA_PERMISSION = 100

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val button = findViewById<Button>(R.id.btnPermission)
        val status = findViewById<TextView>(R.id.tvStatus)

        button.setOnClickListener {

            if (ContextCompat.checkSelfPermission(
                    this,
                    Manifest.permission.CAMERA
                ) != PackageManager.PERMISSION_GRANTED
            ) {

                ActivityCompat.requestPermissions(
                    this,
                    arrayOf(Manifest.permission.CAMERA),
                    CAMERA_PERMISSION
                )

            } else {

                status.text = "Camera Permission Already Granted"
            }
        }
    }

    override fun onRequestPermissionsResult(
        requestCode: Int,
```

```
        permissions: Array<out String>,
        grantResults: IntArray
    ) {
        super.onRequestPermissionsResult(
            requestCode,
            permissions,
            grantResults
        )

        if (requestCode == CAMERA_PERMISSION) {

            if (grantResults.isNotEmpty() &&
                grantResults[0] == PackageManager.PERMISSION_GRANTED
            ) {

                findViewById<TextView>(R.id.tvStatus).text =
                    "Camera Permission Granted"

            } else {

                findViewById<TextView>(R.id.tvStatus).text =
                    "Camera Permission Denied"
            }
        }
    }
}
```

Output

